

Experiment 1- Write the python program to solve 8-Puzzle problem

Aim:

To develop a Python program to solve the 8-Puzzle problem using the Breadth First Search (BFS) algorithm and obtain the sequence of moves from the initial state to the goal state.

Algorithm:

Step 1

Define the initial state and goal state.

Step 2

Create a queue and insert the initial state.

Step 3

Maintain a visited set to avoid revisiting states.

Step 4

Remove a state from the queue.

Step 5

Check whether the current state is the goal state.

Step 6

If the goal is reached, display the solution path.

Step 7

Otherwise, generate all possible child states by moving the blank tile.

Step 8

Add unvisited child states to the queue.

Step 9

Repeat Steps 4–8 until the goal state is found.

Program:

```
from collections import deque
```

```
# Goal State
```

```
GOAL = ((1, 2, 3),  
        (4, 5, 6),  
        (7, 8, 0))
```

```
# Find the position of the blank tile (0)
```

```
def find_blank(state):
```

```

    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Generate all possible moves
def get_neighbors(state):
    x, y = find_blank(state)

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    neighbors = []

    for dx, dy in directions:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            temp = [list(row) for row in state]

            temp[x][y], temp[nx][ny] = temp[nx][ny], temp[x][y]

            neighbors.append(tuple(tuple(row) for row in temp))

    return neighbors

# BFS Algorithm
def solve_puzzle(start):
    queue = deque([(start, [])])
    visited = set()

    while queue:
        current, path = queue.popleft()

        if current == GOAL:
            return path + [current]

        if current in visited:
            continue

        visited.add(current)

        for neighbor in get_neighbors(current):
            if neighbor not in visited:
                queue.append((neighbor, path + [current]))

    return None

# Initial State
start_state = ( (1, 2, 3),
                 (4, 0, 6),
                 (7, 5, 8) )

# Solve the Puzzle
solution = solve_puzzle(start_state)

```

```

# Display Solution
if solution:
    print("Solution Found!\n")

    for step_no, state in enumerate(solution):
        print("Step", step_no)
        for row in state:
            print(row)
        print()
else:
    print("No Solution Exists")

```

Sample Output:

```

Solution Found!

Step 0
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

Step 1
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

Step 2
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

Goal Reached

```

Result:

The 8-Puzzle problem was successfully solved using the Breadth First Search (BFS) algorithm, and the sequence of moves from the initial state to the goal state was obtained.